

Convolution

Thomas Saurel

22 février 2026

<vers le dépôt Github>

Table des matières

1	PNG file format	1
2	Discrete convolution	3
3	C implementation	5
4	Kernels	6
5	Some results	9
6	Bibliography	10

1 PNG file format

This program uses the **PNG (Portable Network Graphics)** extension as input because it is one of the most common formats used to save images; it supports transparency and uses lossless compression. Unlike formats such as JPEG, which apply lossy compression and may introduce visible artifacts, PNG preserves the exact original image data after compression and decompression. This makes it particularly great for applications where image integrity is essential, such as scientific visualization, medical imaging and graphics containing text or sharp edges. Furthermore, PNG supports different color types (grayscale, truecolor, indexed-color, grayscale with alpha, and truecolor with alpha), which makes it flexible for a wide range of use cases.

A PNG file is composed of at least :

1. A **signature** : $(137\ 80\ 78\ 71\ 13\ 10\ 26\ 10)_2$ or $(89\ 50\ 4E\ 47\ 0D\ 0A\ 1A\ 0A)_{16}$
2. An **IHDR** chunk : width, height, bit depth, color type, compression method, filter method, interlace method

3. An **IDAT** chunk : data (can appear multiple times)
4. An **IEND** chunk : end of the data

The **signature** is always the first 8 bytes of the file and identifies the file as a PNG image. It also helps detect file transfer errors and prevents incorrect interpretation of the file format.

The **IHDR** chunk contains critical metadata describing the image. The width and height define the image dimensions in pixels. The bit depth specifies the number of bits per sample. The color type determines how pixel data should be interpreted (grayscale, RGB, RGBA, ...). The filter method specifies the pre-processing applied to image data before compression, improving compression efficiency. The interlace method indicates whether the image uses no interlacing or the Adam7 interlacing scheme, which allows progressive image display.

The **IDAT** chunk (or multiple IDAT chunks) contains the actual compressed image data. Even though there may be several IDAT chunks, their data fields are concatenated to form a single continuous compressed data stream. This design allows flexibility in how encoders write image data while maintaining a consistent decoding process.

The **IEND** chunk marks the end of the PNG file. It does not contain any data and simply signals that no further chunks follow. Its presence is mandatory and ensures proper file termination.

Each chunk is composed of :

1. **Length** : 4 bytes
2. **Type** : 4 bytes
3. **Data** : variable
4. **CRC** : 4 bytes

The **Length** field specifies the number of bytes in the Data field and does not include the Type or CRC fields. The **Type** field is a four-character ASCII code that identifies the purpose of the chunk (e.g., IHDR, IDAT, IEND). The **Data** field contains the actual information associated with the chunk, whose structure depends on the chunk type. Finally, the **CRC** (Cyclic Redundancy Check) is used for error detection ; it is calculated over the Type and Data fields to verify data integrity during reading.

PNG files may also contain optional chunks such as PLTE (palette), tEXt (textual metadata), or tRNS (transparency information). These optional chunks enhance the flexibility of the format while remaining fully backward compatible with decoders that do not interpret them.

I used the libpng (png.h) library to implement the algorithm in C. You can find the manual [here](#) . The libpng library provides a high-level API for reading and writing PNG files, handling low-level details such as chunk parsing, decompression, filtering, and CRC verification.

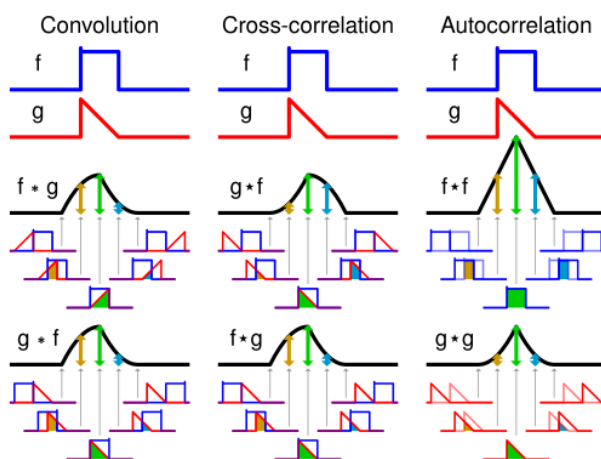
This program does not work if the extension of the file was improperly changed to png, such as renaming a jpg file to png, as the header will not be correctly set up. In such cases, although the filename may indicate a PNG format, the internal structure and signature will not match the required PNG specification. Since the program verifies the file signature and expects valid PNG chunks, it will fail when encountering incompatible file headers or corrupted structures.

2 Discrete convolution

The convolution of two real-valued functions f and g is defined as :

$$(f * g)(x) := \int_{\mathbb{R}} f(x-t)g(t) dt = \int_{\mathbb{R}} f(t)g(x-t) dt$$

This definition comes from continuous signal processing and describes how one function is slid over another. For each position x , we compute the integral of the product between one function and a shifted version of the other. Convolution measures how much two functions overlap as one is translated. It is widely used in physics, probability theory, signal processing, and image analysis because it provides a natural way to model filtering and linear time-invariant systems. It's a very import concept for the Fourier analysis theory.



Example of convolutions (from Wikipedia)

In digital image processing, we cannot work with continuous functions but with discrete data. An image is represented as a finite two-dimensional grid of pixels, so the integral becomes a finite sum. This leads to the definition of **discrete convolution**. Instead of integrating over $\mathbb{R}$, we sum over a small neighbourhood around each pixel.

In the context of image convolution, it takes an original image and a **kernel/filter**. A kernel is a small matrix (I mostly used 3x3, 5x5, and 7x7 matrices) that describes the modifications made to the original image. It must be odd-sized to center it on the desired pixel.

The kernel acts as a set of weights applied to neighbouring pixels. The central value of the kernel corresponds to the current pixel, while the surrounding values determine how much influence neighbouring pixels have on the result. Odd dimensions such as 3×3 or 5×5 are required so that there is a well-defined center element aligned with the processed pixel.

For example, the **Gaussian Blur 5x5** kernel :

$$K_{GB_5} = \frac{1}{273} \begin{pmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{pmatrix}$$

This kernel approximates a discrete Gaussian distribution. The largest coefficient is located at the center, meaning the current pixel contributes the most to the final value. The surrounding coefficients decrease symmetrically with distance from the center, producing a smoothing effect. The normalization factor $\frac{1}{273}$ ensures that the sum of all coefficients equals 1, which preserves the overall brightness of the image after filtering.

If we define :

1. A kernel of size $(2m + 1) \times (2n + 1)$,
2. $I(x, y)$ the value of a pixel (R,G,B or A) at the coordinates (x, y) ,
3. $K(i, j)$ the value of the kernel at the (i, j) coordinates,

we get the equality :

$$P(x, y) := \sum_{i=-m}^m \sum_{j=-n}^n I(x + i, y + j) K(i, j)$$

where $P(x, y)$ is the new value of the pixel (x, y) after convolution.

This formula represents the discrete two-dimensional convolution. For each pixel (x, y) in the image, we place the center of the kernel on that pixel. Then, we multiply each neighbouring pixel value by the corresponding kernel coefficient

and sum all these products. The result becomes the new value of the pixel. We then repeat the process on each of the four channels to get an *RGBA* value.

In practice, we must be careful of the image borders, since some neighbours may fall outside the image domain. Common strategies include replacing outside pixels with zero or repeating edge value. The choice of border handling method can slightly affect the final visual result.

Different kernels produce different visual effects. For instance, averaging kernels create blur, Laplacian kernels enhance edges, and Sobel kernels detect gradients. Therefore, convolution provides a unified mathematical framework for many fundamental image processing operations.

3 C implementation

You first need to enter the command **make** in a terminal in the folder that contains the Makefile.

This command compiles all the source files and generates the executable file (named **main** in this project). The Makefile automatically handles compilation flags, dependencies, and linking with the required external libraries (such as **libpng**). If the compilation succeeds, the executable will appear in the same directory. If an error occurs, the compiler will display a detailed message indicating which file or function caused the issue.

To use the program, use the following command :

```
./main INPUT_LOCATION OUTPUT_LOCATION KERNEL_ID
```

- **INPUT_LOCATION** is the relative or absolute path to the PNG image you want to process.
- **OUTPUT_LOCATION** is the path where the filtered image will be saved.
- **KERNEL_ID** selects the convolution kernel applied to the image.

The program reads the input PNG file, extracts its pixel data (RGBA channels), applies the selected convolution kernel independently to each channel, and writes the processed image back to disk as a new PNG file.

KERNEL_ID represents the kernel that you want to use :

- **IDENTITY** : 0
- **BLUR 3x3** : 1
- **BLUR 5x5** : 2
- **BLUR 7x7** : 3
- **GAUSSIAN BLUR 3x3** : 4
- **GAUSSIAN BLUR 5x5** : 5
- **GAUSSIAN BLUR 7x7** : 6
- **SHARP** : 7
- **ULTRA SHARP** : 8
- **SOBEL X** : 9
- **SOBEL Y** : 10
- **LAPLACIAN** : 11

- EMBOSS : 12
- OUTLINE : 13
- CHECKER 3x3 : 14
- CHECKER 9x9 : 15

Each kernel corresponds to a predefined matrix stored in the source code. For example, blur kernels compute the average of neighbouring pixels, Gaussian blur kernels apply weighted smoothing, Sobel kernels detect horizontal or vertical gradients, and sharpening kernels enhance intensity differences between neighbouring pixels. The identity kernel (n°0) simply returns the original image without modification and is useful for testing.

Of course, you can define your own kernel, for example :

```
float MY_KERNEL[9] = {
    -2.f, -1.f, 0.f,
    -1.f, 1.f, 1.f,
    0.f, 1.f, 2.f
};
```

I did not find any issues or memory leaks (checked with Valgrind). However, some kernels may not work well on certain images. For example, the outline kernel is sensitive to contrast : if the image has low contrast or lacks a transparent border, the kernel might produce near-zero results. It works best with images that have high contrast or a translucent background.

4 Kernels

Here is a list of all default kernels implemented. Each kernel corresponds to a specific visual effect obtained by modifying the local neighbourhood of every pixel. The size of the kernel determines how many neighbouring pixels influence the result : larger kernels generally produce stronger or smoother effects but require more computation time.

1. Identity : $\begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$: Does not change the image

The identity kernel leaves the image unchanged because only the central pixel has a non-zero coefficient. All neighbouring pixels are multiplied by zero, so the convolution result is exactly the original pixel value. This kernel is mainly useful for debugging or verifying that the convolution implementation works correctly without altering the image.

2. Blur : $\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}, \frac{1}{25} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}, \frac{1}{49} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}$

Averaging blur

$$\sum_{i=1}^n \sum_{j=1}^n a_{ij} = 1 \implies \text{Keep the same average intensity}$$

These blur kernels compute the simple arithmetic mean of the neighbouring pixels. Each coefficient has the same value, meaning every neighbour contributes equally to the final result. The normalization factor ($\frac{1}{9}$, $\frac{1}{25}$, or $\frac{1}{49}$) ensures that the sum of all coefficients equals 1, preserving the global brightness of the image. Increasing the kernel size increases the smoothing effect, as more neighbouring pixels are taken into account. However, large averaging kernels may significantly reduce image sharpness and fine details.

$$3. \text{ Gaussian Blur : } \frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}, \frac{1}{273} \begin{pmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{pmatrix},$$

$$\frac{1}{1003} \begin{pmatrix} 0 & 0 & 1 & 3 & 1 & 0 & 0 \\ 0 & 3 & 13 & 22 & 13 & 3 & 0 \\ 1 & 13 & 59 & 97 & 59 & 13 & 1 \\ 2 & 22 & 97 & 159 & 97 & 22 & 2 \\ 1 & 13 & 59 & 97 & 59 & 13 & 1 \\ 0 & 3 & 13 & 22 & 13 & 3 & 0 \\ 0 & 0 & 1 & 3 & 1 & 0 & 0 \end{pmatrix}$$

Smooth image with Gaussian weights

Weights are given by the gaussian function $G(x, y) = \frac{1}{2\pi\sigma} e^{-\frac{x^2+y^2}{2\sigma}}$

Unlike the simple averaging blur, Gaussian blur assigns larger weights to pixels closer to the center and smaller weights to those farther away. This produces a more natural and visually pleasing smoothing effect. The Gaussian distribution ensures radial symmetry and avoids abrupt transitions. The parameter σ controls the spread of the blur : a larger σ produces stronger smoothing. Gaussian blur is widely used as a preprocessing step for noise reduction and edge detection.

$$4. \text{ Sharp : } \begin{pmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{pmatrix} : \text{Enhances edges slightly}$$

This sharpening kernel increases the contribution of the central pixel relative to its neighbours. As a result, intensity differences between adjacent pixels become more pronounced, making edges appear clearer. Mild sharpening enhances fine details without introducing too much noise.

$$5. \text{ Ultra Sharp : } \begin{pmatrix} 0 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 0 \end{pmatrix} : \text{Stronger edge enhancement}$$

This kernel significantly amplifies the central pixel value. Compared to the standard sharpening filter, it produces a much stronger contrast enhancement. However, it may also amplify noise and create unnatural-looking results if applied excessively. Such aggressive sharpening is generally used for experimental purposes or for images that are very blurred initially.

$$6. \text{ Sobel X : } \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} : \text{Edge detection (left) in horizontal direction}$$

The Sobel X kernel highlights vertical edges, meaning transitions where in-

tensity changes from left to right. Large positive or negative values appear where there is a strong horizontal gradient. This filter is commonly used in edge detection algorithms, and its structure combines differentiation with slight smoothing (due to the middle row coefficients -2 and 2).

7. Sobel Y : $\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$: Edge detection (top) in vertical direction

The Sobel Y kernel highlights horizontal edges, meaning transitions from top to bottom. As with Sobel X, the weighting gives more importance to the central pixels, which provides a smoothing effect while computing the gradient. In practice, Sobel X and Sobel Y are often combined to compute the gradient magnitude :

$$|\nabla I| = \sqrt{(G_x)^2 + (G_y)^2}$$

where G_x and G_y are the results of the Sobel X and Sobel Y convolutions. This produces a more complete edge map of the image.

8. Laplacian : $\begin{pmatrix} -1 & 2 & -1 \\ 2 & -4 & 2 \\ -1 & 2 & -1 \end{pmatrix}$: Detects edges in all directions

The Laplacian kernel approximates the second derivative of the image intensity. Unlike Sobel filters, which detect edges in specific directions, the Laplacian responds to intensity changes in all directions simultaneously. It highlights regions where the intensity changes rapidly, regardless of orientation. Because it is a second-order derivative operator, it is more sensitive to noise ; therefore, it is often applied after a smoothing step such as Gaussian blur.

9. Emboss : $\begin{pmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix}$: Creates a 3D emboss effect

The emboss kernel enhances edges while introducing an artificial lighting effect. One side of the edge becomes brighter and the opposite side darker, simulating a shadow and highlight. This produces an engraved appearance, giving the impression that the image has depth. The effect depends on the orientation of the kernel and can simulate light coming from a specific direction.

10. Outline : $\begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$: Highlights strong edges

The outline kernel emphasizes regions with strong local intensity variation. The large positive central coefficient combined with negative surrounding coefficients causes flat regions to cancel out (sum close to zero), while edges produce high absolute values. This filter is particularly effective when applied to high-contrast images. However, in low-contrast regions, the resulting values may be small and produce faint edges.

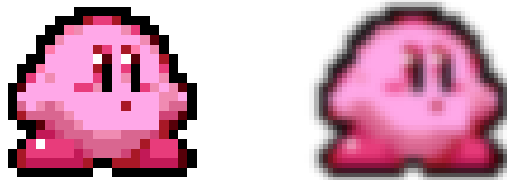
$$11. \text{ Checker : } \begin{pmatrix} 1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & 1 \end{pmatrix}, \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 & -1 & -1 & -1 & 1 & 1 & 1 \\ 1 & 1 & 1 & -1 & -1 & -1 & 1 & 1 & 1 \\ 1 & 1 & 1 & -1 & -1 & -1 & 1 & 1 & 1 \\ -1 & -1 & -1 & 1 & 1 & 1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 1 & 1 & 1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 1 & 1 & 1 & -1 & -1 & -1 \\ 1 & 1 & 1 & -1 & -1 & -1 & 1 & 1 & 1 \\ 1 & 1 & 1 & -1 & -1 & -1 & 1 & 1 & 1 \\ 1 & 1 & 1 & -1 & -1 & -1 & 1 & 1 & 1 \end{pmatrix}$$

Produce a checkerboard pattern

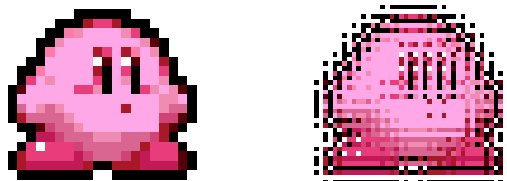
These checker kernels alternate positive and negative coefficients to create periodic intensity variations. When applied to an image, they emphasize alternating regions and can generate a checkerboard-like texture. The larger 9×9 version produces a more pronounced and structured pattern due to the repetition of positive and negative blocks. Such kernels are mainly experimental and demonstrate how convolution can create artificial textures rather than simply enhancing or detecting features.

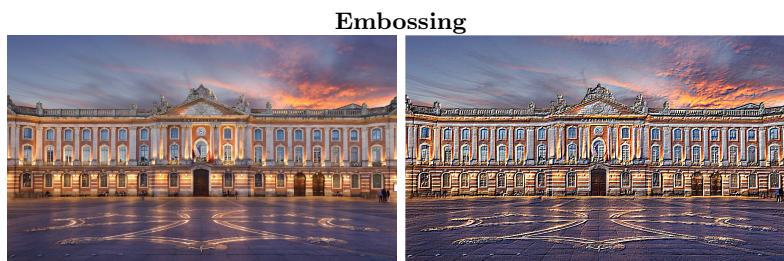
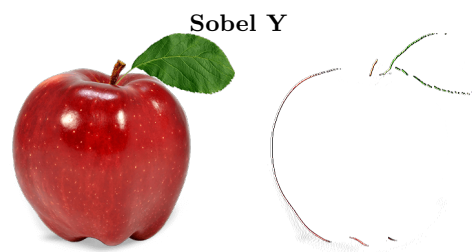
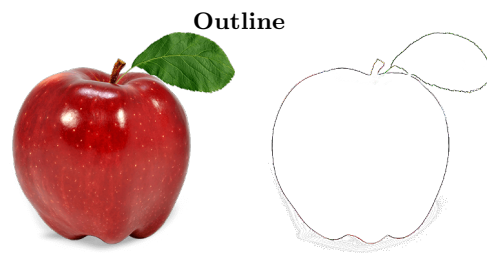
5 Some results

Gaussian Blur



Checker





6 Bibliography

Main online references :

1. Youtube : But what is a convolution? : 3Blue1Brown
2. Wikipedia : PNG
3. Wikipedia : Kernel
4. Wikipedia : Convolution
5. libpng.org
6. Multiple discussions on Stack Overflow (used for debugging and implementation)